

Тестовое задание

frontend · react · typescript · дедлайн: 2-3 дня

Задача

Реализовать дашборд для мониторинга орг-структуры компании. Структура иерархическая: дивизионы → отделы → команды. У каждого узла — численность сотрудников, бюджет и метрика эффективности. Нужен интерактивный просмотр дерева и аналитическая таблица с агрегированными показателями.

Стек

React, Vite, TypeScript.

Будет плюсом

Использование styled-components для стилизации.

Нефункциональные требования

- Надёжность: обработка состояний загрузки, ошибки и пустого ответа; корректная отмена запросов при размонтировании компонента.
- Эффективность обращения к API: кэширование ответов, избегание лишних запросов (stale-while-revalidate или аналог); инвалидация кэша только при реальном изменении данных.

Обязательно: AI-инструменты (Claude, Cursor, Copilot и др.) в процессе разработки. В README опишите: что генерировали, что переписали руками и почему.

Mock API

Реализуйте собственный сервер. GET /api/org-tree возвращает плоский массив:

```
{ id, name, parentId, headcount, budget, performance (0-100), updatedAt }
```

Минимум 40 узлов, не менее трёх уровней вложенности.

Этапы

Каждый этап — отдельный коммит с тегом step/N.

01 FOUNDATION

Проект, API, дерево

- Scaffold: Vite + React + TypeScript, абсолютные импорты
- Схема API-ответа валидируется на клиенте; невалидный ответ — ошибка
- Запрос через слой кэширования, stale time 5 секунд
- Интерактивное дерево: раскрытие/скрытие ветвей, второй уровень открыт по умолчанию
- Каждый узел показывает name, headcount и цветовой индикатор performance
- Состояния: загрузка, ошибка, пустой ответ; никакого inline-CSS

02 CORE

Аналитическая таблица

- Переключатель «Дерево / Таблица» (или split-view $\geq 1280\text{px}$)
- Столбцы: Подразделение, Уровень, Всего сотрудников, Бюджет суммарный, Средняя

эффективность

- Суммарные показатели включают данные узла и всех его потомков; средняя эффективность — взвешенная по headcount
- Сортировка по любому столбцу, двойной клик — обратная
- Фильтр по названию: real-time, дебаунс 250мс
- Клик по строке выделяет узел в дереве; бюджеты в формате 12 345 678 руб.

Агрегация считается один раз после загрузки данных и мемоизируется.

03 POLISH

Live-обновления и UX

- Сервер поддерживает обновления в реальном времени. Клиент реализует один из вариантов: WebSocket, SSE или эффективный поллинг (с учётом частоты изменений, backoff и минимизации лишних запросов).
- Клиент применяет патч без полного рефетча; обновлённые ячейки — fade-out ~1.5с
- Пересчёт агрегатов только для затронутого узла и его предков
- Индикатор соединения в шапке; экспоненциальный backoff при обрыве
- Keyboard navigation по таблице: стрелки, Home/End, Enter
- Анимация раскрытия дерева через height transition; prefers-reduced-motion уважается

04 BONUS

Production + AI-фича

- Docker: docker-compose up поднимает клиент и сервер; конфиг через .env
- Nginx: проксирует API, отдаёт статику с gzip; production-сборка ≤200 КБ gzip
- AI-поиск: строка поиска принимает естественный язык; ответ — структурированный фильтр на клиенте; fallback — текстовый поиск

Что сдавать

- Публичный репозиторий или архив с исходным кодом; каждый этап — отдельный коммит с тегом step/N
- README: запуск одной командой + раздел «AI в разработке» (обязательно)
- Unit-тест на функцию агрегации; скриншоты или GIF финального результата
- docs/architecture.md — слои приложения, поток данных от API до UI
- docs/data-model.md — дерево, алгоритм агрегации, контракт WebSocket-патча
- docs/adr/NNN-название.md — ADR для нетривиальных решений (контекст, решение, альтернативы, последствия)

Не нужно: авторизацию, БД, UI-библиотеки (MUI, Ant и т.п.). Всё, что не оговорено явно — на ваше усмотрение.